00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D

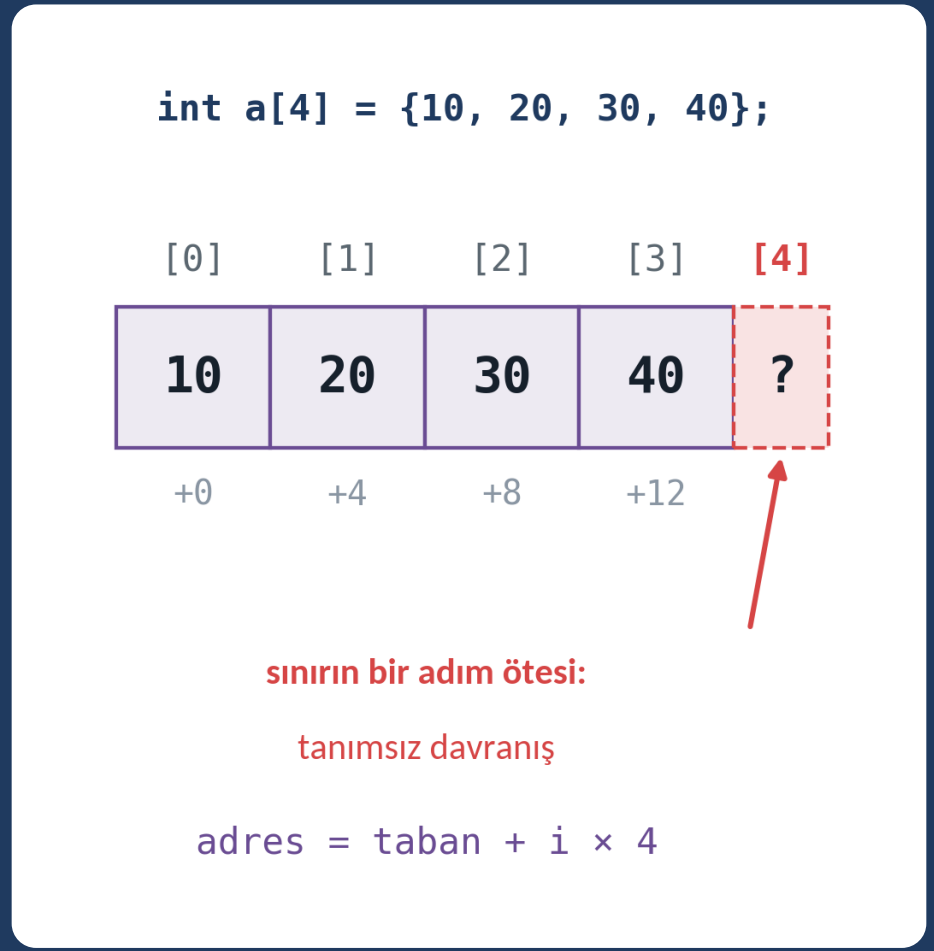
ALGORİTMA VE PROGRAMLAMA – I

HAFTA 10

Diziler I: Bitişik Bellek ve Tek Boyutlu Diziler

Bildirim ve ilk değer · indis ve adres hesabı · dizi desenleri ve frekans · fonksiyonlara aktarma · sınır dışı erişim ve Heartbleed

MODÜL V · VERİ TOPLULUKLARI VE BELLEK



Dr. Öğr. Üyesi Mehmet Sevri · RTEÜ Bilgisayar Mühendisliği · 2026–2027 Güz

14 HAFTALIK YOLCULUK

Yol haritasında neredeyiz?

H1	H2	H3	H4	H5	H6	H7
Bilgisayar ve Program	Algoritmik Düşünme	Değişkenler ve Veri Tipleri	Operatörler ve İfadeler	Karar Yapıları	Döngüler I	Döngüler II + Ara Proje
0x00	0x01	0x02	0x03	0x04	0x05	0x06

H8	H9	H10	H11	H12	H13	H14
Fonksiyonlar I	Kapsam, Yığın, Özyineleme	**Diziler I**	Diziler II: Matrisler	İşaretçiler ve Bellek	Karakter Dizileri	Yapılar, Dosyalar, Final
0x07	0x08	0x09	0x0A	0x0B	0x0C	0x0D

Vize: H1–H7 · Ara proje: H7 · Final projesi: H14

GERİ ÇAĞIRMA

Geçen haftadan: üç soru (H9)

S1 · static: main `say()`'ı iki kez boşa çağırır, üçüncüsünün sonucunu yazdırır. Ne yazdırılır?

quiz/H10_Q01_static_sayac.c · satır 7–12

```c
int say(void)
{
    static int n = 0;
    n++;
    return n;
}
```

S2 · çağrı yığını: main `m1` yazar, `a()`'yı çağırır, sonra `m2`. Çıktı sırası? En çok kaç çerçeve?

quiz/H10_Q02_cagri_yigini.c · satır 12–17

```c
void a(void)
{
    printf("a1 ");
    b();
    printf("a2 ");
}
```

S3 · özyineleme (`H10_Q03_basamak_toplami.c`): `topla(0) = 0`, `topla(n) = n % 10 + topla(n / 10)`.
`topla(2026)` kaç döndürür? Temel durum hangisi; kaç çağrı yapılır?

Geçen haftadan: yanıtlar (gerçek çıktılar)

Bu hafta yığını yeniden göreceğiz: fonksiyon içindeki **diziler yığında** durur.

S1 → 3

static int n bir kez sıfırlanır; her çağrı bir artırır. Ömür statik, kapsam blok.

S2 → m1 a1 b a2 m2

b çalışırken yığında main, a ve b: **3 çerçeve**. Her dönüş, çağrının yapıldığı satıra gelir.

S3 → 10

6 + 2 + 0 + 2; temel durum `n == 0`. **5 çağrı**: (2026) → (202) → (20) → (2) → (0).

GERÇEK ÇALIŞTIRMALAR (S3: GDB KESME NOKTASIYLA ÇAĞRI DİZİSİ)

```
$ ./q01
3
$ ./q02
m1 a1 b a2 m2
$ gdb -q -batch -x iz.gdb ./q03      # topla her çağrıldığında n
topla(2026)  topla(202)  topla(20)  topla(2)  topla(0)
10
```

Haftanın Büyük Sorusu

Yüz öğrencinin notunu yüz ayrı değişkende mi tutacağız ve dizinin sınırını bir adım aşmak neden bir güvenlik felaketine dönüşebilir?



`not1, not2, …, not100?  ya da  int notlar[100];`

- Mini-OBS v0.9 notları **saklamıyordu**: “ortalamanın üstündekiler kim?” sorusunu yanıtlayamıyordu.
- Dizi: aynı tipten değerler, **tek ad**, **bitişik bellek**, **indisle** erişim.
- Ama C, dizinin sınırını sizin yerinize **denetlemez**: 2014’te tek bir eksik denetim, popüler HTTPS sunucularının tahminen %24–55’ini açık bıraktı.

GERÇEK DÜNYADAN

7 Nisan 2014

Heartbleed (CVE-2014-0160): uzunluk alanına güvenmenin bedeli

- TLS Heartbeat: bir taraf bir **yük** ve yükün **uzunluğunu** gönderir, karşı taraf aynı yükü geri yollar (RFC 6520, Şubat 2012; OpenSSL 1.0.1, 14 Mart 2012).
- OpenSSL, **bildirilen** uzunluğu gerçek yükle **karşılaştırmıyordu**: yükün ardındaki bellek yanıta kopyalanıyordu; istek başına en çok ~64 KB.
- Açığı Neel Mehta (Google Security) ve Codenomicon birbirinden bağımsız buldu; 7 Nisan 2014'te 1.0.1g ile duyuruldu. Etkilenen: 1.0.1–1.0.1f.
- Duyuru anında popüler HTTPS sitelerinin tahminen **%24–55**'i açıktı; özel anahtarlar okunabiliyordu.

Durumeric vd. (2014); OpenSSL Project (2014); NIST NVD (2014)

Benzetim: uzunluğa güvenen kopyalama (kendi dizimizin içinde)

Sunucu belleği (benzetimde TEK bir char dizisi):



yük (3 bayt)

bitişik bellekteki başka veriler (oturum anahtarı, parola ...)

İstek 1: "KUS", uzunluk = 3



`yanıt: "KUS" (3 bayt)`

İstek 2: "KUS", uzunluk = 31 (yalan!)



`hatalı yanıt: "KUS|oturum=7f3a91|parola=Rize53" (31 bayt)`

Düzeltme: bildirilen uzunluk > gerçek yük ise isteği yanıtlama (yanit_duzeltilmis → "istek atıldı").
Gerçek açık: uzunluk alanı 2 bayt → istek başına en çok 64 KB bellek (OpenSSL 1.0.1–1.0.1f; düzeltme 1.0.1g).

ÖĞRENME ÇIKTILARI

Bu hafta sonunda neler yapabileceksiniz?

ÖÇ 10.1	Tek boyutlu dizileri bildirir, ilk değer atar ve indisler üzerinden erişir.
ÖÇ 10.2	Dizilerin bellekte bitişik yerleşimini ve indis–adres ilişkisini (taban adres + i × eleman boyutu) açıklar.
ÖÇ 10.3	Temel dizi işleme desenlerini (toplam, ortalama, en büyük ve konumu, sayma, frekans, ters çevirme, kaydırma, kopyalama) uygular.
ÖÇ 10.4	Dizileri boyut bilgisiyle birlikte fonksiyonlara aktarır.
ÖÇ 10.5	Dizi sınırı aşımının (out-of-bounds) sonuçlarını ve güvenlik riskini açıklar.

01

Kavramlar: dizi ve bellek

Neden dizi · bildirim · adres hesabı · sizeof · fonksiyonlar · sınır dışı erişim

Neden dizi? Beş değişken mi, bir dizi mi?

H10_K01_ayri_degiskenler.c · satır 17–24

```c
17      int not1 = 72, not2 = 85, not3 = 64, not4 = 90, not5 = 58;
18      double ort = (not1 + not2 + not3 + not4 + not5) / 5.0;
19      printf("Ayrı değişkenler : ortalama %.2f, üstündekiler:", ort);
20      if (not1 > ort) { printf(" %d", not1); }
21      if (not2 > ort) { printf(" %d", not2); }
22      if (not3 > ort) { printf(" %d", not3); }
23      if (not4 > ort) { printf(" %d", not4); }
24      if (not5 > ort) { printf(" %d", not5); }
```

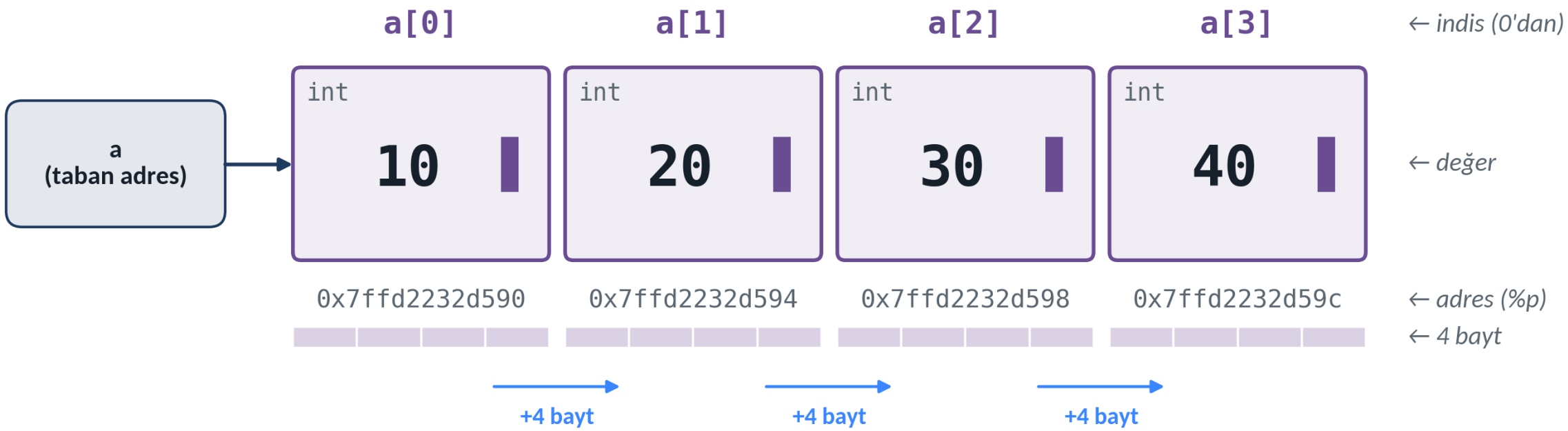
H10_K01_ayri_degiskenler.c · satır 35–39

```c
35      for (int i = 0; i < N; i++) {
36          if (notlar[i] > ortalama) {
37              printf(" %d", notlar[i]);
38          }
39      }
```

İki yöntem de aynı çıktıyı verir: “Diziyle : ortalama 73.80, üstündekiler: 85 90”. 100 öğrencide: 100 değişken ve 100 if — ya da yalnızca N = 100.

Dizi = numaralı dolap şeridi

`int a[4] = {10, 20, 30, 40};` → 4 kutu × 4 bayt = 16 bayt, art arda (bitişik)



Bildirim ve ilk değer listeleri

H10_K02_bildirim_ilk_deger.c · satır 17–21

```c
17      int a[5] = {1, 2, 3, 4, 5};          /* tam liste */
18      int b[5] = {1, 2};                   /* kısmi: kalanlar 0 */
19      int c[5] = {0};                      /* hepsi 0 */
20      int d[] = {4, 8, 15, 16, 23, 42};    /* boyut listeden: 6 */
21      int e[6] = {[2] = 7, [5] = 9};       /* C99: belirlenmiş ilk değer */
```

GERÇEK ÇIKTI (H10_K02)

```
a (tam liste)  : 1 2 3 4 5
b (kısmi)      : 1 2 0 0 0
c ({0})        : 0 0 0 0 0
d (boyut: 6)   : 4 8 15 16 23 42
e ([2], [5])   : 0 0 7 0 0 9
g (global)     : 0 0 0 0 0
```

- Kısa liste: **kalanlar 0**
- `= {0}`: hepsini sıfırla
- Boyutsuz: listeden (6)
- Global dizi: kendiliğinden 0
- Yerel ilk değersiz dizi: **belirsiz** (HA2)

TAHMİN ET · KISMİ İLK DEĞER

Program ne yazdırır?

quiz/H10_Q11_kismi_ilk_deger.c · satır 7–12

```c
int main(void)
{
    int a[5] = {3};
    printf("%d %d\n", a[0], a[4]);
    return 0;
}
```

A `3 3`

B `3 0`

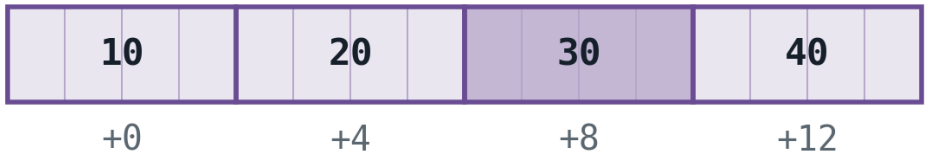
C `3` ve belirsiz bir değer

D Derleme hatası

Adres hesabı: taban + i × sizeof(eleman)

adres(x[i]) = taban + i × sizeof(x[0])

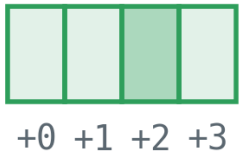
```
int    a[4] · sizeof = 4 · taban 0x7ffd2232d590 → &a[2] = taban + 2×4 = 0x7ffd2232d598
```




```
double d[4] · sizeof = 8 · taban 0x7ffd2232d5a0 → &d[2] = taban + 2×8 = 0x7ffd2232d5b0
```




```
char   c[4] · sizeof = 1 · taban 0x7ffd2232d5c4 → &c[2] = taban + 2×1 = 0x7ffd2232d5c6
```



'A' 'B' 'C' 'D': her biri 1 bayt

Gerçek adresler: iki çalıştırma, aynı farklar

1. ÇALIŞTIRMA (CHAR DİZİSİNİN ADRESLERİ DERS NOTUNDA)

```
i=0  &a[i]=0x7ffd2232d590  &d[i]=0x7ffd2232d5a0
i=1  &a[i]=0x7ffd2232d594  &d[i]=0x7ffd2232d5a8
i=2  &a[i]=0x7ffd2232d598  &d[i]=0x7ffd2232d5b0
i=3  &a[i]=0x7ffd2232d59c  &d[i]=0x7ffd2232d5b8
```

2. ÇALIŞTIRMA: TABAN DEĞİŞTİ (ASLR), FARKLAR AYNI

```
i=0  &a[i]=0x7fffb2113a60  &d[i]=0x7fffb2113a70
i=1  &a[i]=0x7fffb2113a64  &d[i]=0x7fffb2113a78
i=2  &a[i]=0x7fffb2113a68  &d[i]=0x7fffb2113a80
i=3  &a[i]=0x7fffb2113a6c  &d[i]=0x7fffb2113a88
```

- `int` adımları 4, `double` adımları 8 bayt · `a[2] = 30, *(a + 2) = 30` → `a[i] ≡ *(a + i)` (ayrıntısı H12)
- GCC bile öyle görür: `a[i] = 0;` hata iletisinde `*(a + (sizetype)((long unsigned int)i * 4))` yazıyor

Döngüyle dolaşma, kapasite ve dolu eleman sayısı

H10_01_dizi_oku_yaz.c · satır 14–22

```
14      int notlar[N];                      /* 5 int: notlar[0] .. notlar[4] */
15
16      printf("%d not girin: ", N);
17      for (int i = 0; i < N; i++) {
18          if (scanf("%d", &notlar[i]) != 1) {   /* &: i. elemanın adresi */
19              printf("Geçersiz girdi.\n");
20              return 1;
21          }
22      }
```

- Standart kalıp `for (int i = 0; i < N; i++)`: tam **N tur**, i = 0 … N − 1
- `scanf`'e adres: `&notlar[i]` (Hata Avcısı 5) · girdi: "72 85 64 90 58"
- **Kapasite ≠ n**: `#define KAPASITE 100` ve dolu eleman sayısı `n`; kullanıcıdan gelen n kapasiteyle denetlenir (Örnek 3)

Eleman sayısını derleyiciye hesaplatmak: sizeof

H10_K04_sizeof.c · satır 13

```c
#define ELEMAN_SAYISI(d) (sizeof(d) / sizeof((d)[0]))   /* parantezler! */
```

GERÇEK ÇIKTI (X86-64 LİNUX, LP64)

```
int    a[10]: sizeof(a) = 40, sizeof(a[0]) = 4, eleman = 10
double b[10]: sizeof(b) = 80, sizeof(b[0]) = 8, eleman = 10
char   c[10]: sizeof(c) = 10, sizeof(c[0]) = 1, eleman = 10
```

- `sizeof(dizi)` = toplam bayt, `sizeof(dizi[0])` = bir eleman → bölüm = **eleman sayısı** (derleme zamanında)
- Makroda parantezler: H9'daki `KARE(x)` dersi
- **Yalnızca dizinin bildirildiği yerde** çalışır: fonksiyon parametresinde 8 / 4 = 2 (Hata Avcısı 3)

Değişken uzunluklu diziler (VLA): kısa bir not

- C99: **zorunlu** · C11/C17: **isteğe bağlı**; desteklemeyen derleyici `__STDC_NO_VLA__` = 1 tanımlar
- GCC destekler (makro tanımlı değil) · MSVC `/std:c11`, `/std:c17` kipinde **desteklemez** (Microsoft, t.y.)
- Risk: büyük n yığını taşırır (Linux'ta tipik 8 MB, H9) · CERT C ARR32-C: boyutu denetle
- Bu derste **sabit kapasite**; çalışma zamanında boyutlanan güvenli dizi → H12 (`malloc`)

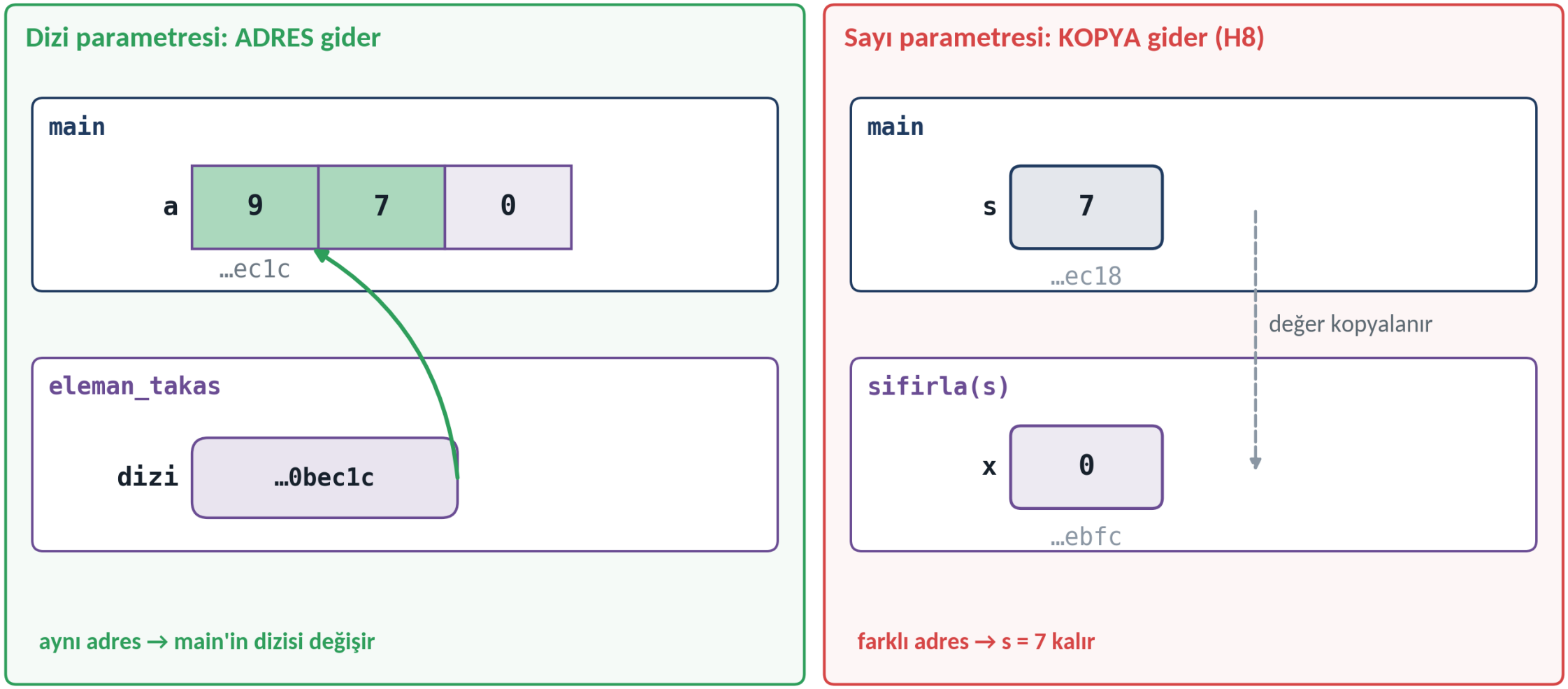
H10_K05_vla.c · satır 26–31

```c
26      int kareler[n];                     /* VLA: boyut n, çalışma zamanında */
27      for (int i = 0; i < n; i++) {
28          kareler[i] = i * i;
29      }
30      printf("kareler[%d] = %d\n", n - 1, kareler[n - 1]);
31      printf("sizeof(kareler) = %zu bayt (çalışma zamanında)\n", sizeof(kareler));
```

GERÇEK ÇIKTI: GİRDİ 6

```
kareler[5] = 25
sizeof(kareler) = 24 bayt (çalışma zamanında)
warning: ISO C90 forbids variable length array ‘kareler’ [-Wvla]   ← -Wvla ile
```

Diziyi fonksiyona aktarmak: ADRES gider, boyut ayrı



Kopyalama ve karşılaştırma: eleman eleman

H10_K09_kopyala_karsilastir.c · satır 23–34

```
23      for (int i = 0; i < N; i++) {       /* kopyalama: eleman eleman */
24          b[i] = a[i];
25      }
26
27      bool esit = true;                   /* karşılaştırma: ilk farkta dur */
28      for (int i = 0; i < N; i++) {
29          if (a[i] != b[i]) {
30              esit = false;
31              break;
32          }
33      }
34      printf("a ile b %s\n", esit ? "eşit" : "farklı");
```

- `b = a;` → **derleme hatası**; `a == b` → adresleri karşılaştırır, hep “farklı” (Hata Avcısı 4) · gerçek çıktı: “a ile b eşit”
- Birleştirme: ikinci dizi `birlesik[N + i]`’ye, birincinin hemen ardına · kütüphane: `memcpy`, `memcmp` (bayt cinsinden)

Paralel diziler: aynı indis = aynı öğrenci

	[0]	[1]	[2]	[3]	[4]
no[]	2301010045	2301010046	2301010047	2301010048	2301010049
vize[]	70.00	45.00	90.00	55.00	60.00
final[]	85.00	48.00	95.00	60.00	72.00
ort[]	79.00	46.80	93.00	58.00	67.20

i = 2: aynı indis = aynı öğrenci

n = 5 (dolu eleman)
KAPASİTE = 100

Risk: silme/sıralama
4 diziye AYNI işlem

H14: struct Ogrenci

Sınır dışı erişim: tanımsız davranış

a[i], taban + i × boyut adresine gider; i dizinin dışındaysa standart hiçbir şey garanti etmez.

Sessiz bozulma

- a[4] = 77 → görünür etki **yok**
- a[-4] = 77 → komşu `once` = 77
- En tehlikelisi: testte görünmez

Durdurma / çökme

- a[6] → yığın koruyucusu durdurur
- a[10] → dönüş adresi: Segmentation fault
- Sonuç derleyiciye ve ayara bağlı

Yakalamak

- C sınır denetimi **yapmaz**
- `-fsanitize=address`: ilk yazmada rapor
- Program her dış indisi denetlemeli

GERÇEK ÇALIŞTIRMALAR: H10_13_SİNİR_DİSİ.C (GCC 13.3, -O0) · AYRINTI ÖRNEK 13

```
a[-4] = 77  → a = {10, 20, 30, 40} once = 77 sonra = 222 · çıkış 0
a[4] = 77   → a = {10, 20, 30, 40} once = 111 sonra = 222 · çıkış 0
a[6] = 77   → *** stack smashing detected *** · çıkış 134
a[10] = 77  → Segmentation fault · çıkış 139
```

KAPUTUN ALTINDA · SEZGİ

Sıralı erişim neden hızlı? Önbellek satırları

Önbellek satırı = 64 bayt (bu makine: getconf) = 16 int → bir ıska, 16 elemanı birden getirir

`sıralı: a[0], a[1], a[2], …`



2 ıska + 30 isabet (her satırda 1 ıska)

`atlamalı: a[0], a[16], a[32], …`



her erişim yeni bir satır → her erişim ıska

ı = ıska (miss): veri RAM'den satır satır gelir. Ölçüm H11'de (matris satır/sütun dolaşma).

Atölye: dizi desen kartları

Toplam / ortalama

`toplam += a[i];` · `(double)toplam / n`

Sayma

`if (a[i] == x) sayac++;` · erken çıkış yok

En büyük + indis

`enb = 0; if (a[i] > a[enb]) enb = i;`

Doğrusal tarama

ilk eşleşmede `break`; yoksa `-1`

İki uç

`i = 0, j = n - 1; i < j` · takas

Frekans

`frekans[deger]++;` · değer = indis

02

Örnek merdiveni

14 örnek: toplamdan Eratosthenes Kalburu’na

Örnek 2 ★: toplam ve ortalama (sizeof ile eleman sayısı)

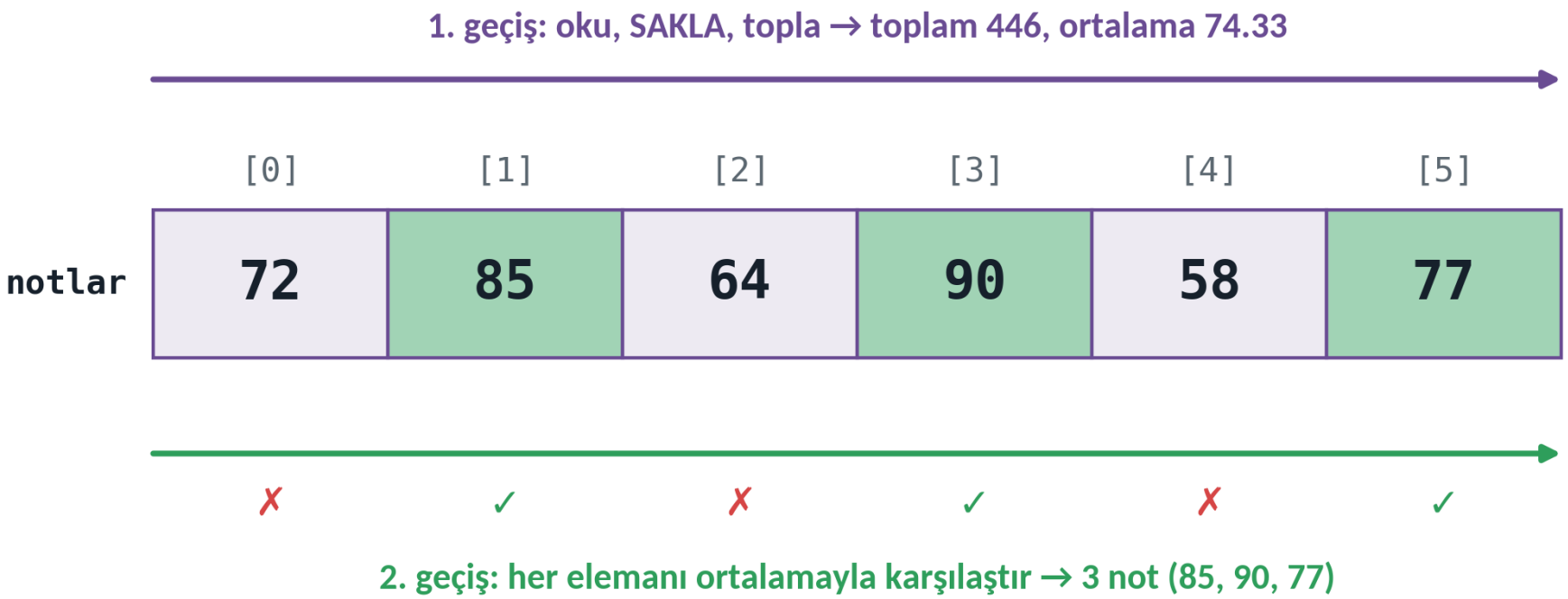
H10_02_toplam_ortalama.c · satır 13–20

```c
13      int notlar[] = {72, 85, 64, 90, 58, 77, 81};   /* boyut listeden: 7 */
14      int n = (int)(sizeof(notlar) / sizeof(notlar[0]));
15
16      int toplam = 0;                      /* toplayıcı: 0'dan başlar */
17      for (int i = 0; i < n; i++) {
18          toplam += notlar[i];
19      }
20      double ortalama = (double)toplam / n;          /* tamsayı bölmesi değil */
```

GERÇEK ÇIKTI

```
Dizinin boyutu: 28 bayt, eleman: 7
Toplam   : 527
Ortalama : 75.29
```

Örnek 3 ★: ortalamanın üstündekiler — iki geçiş



Örnek 4–5 ★★: sayma, en büyük ve indisi

H10_05_en_buyuk_indis.c · satır 16–25

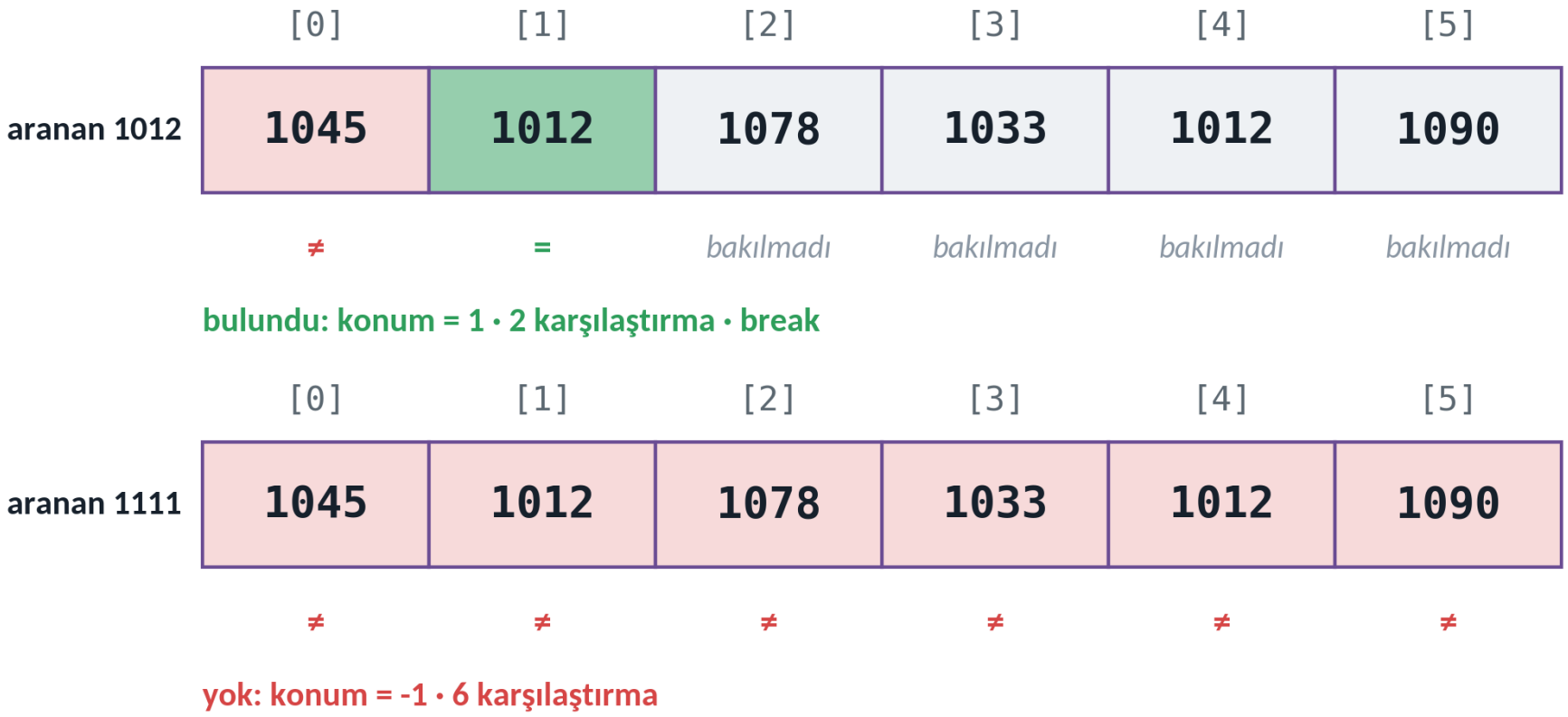
```
16      int enb = 0;                        /* en büyüğün İNDİSİ */
17      int enk = 0;                        /* en küçüğün İNDİSİ */
18      for (int i = 1; i < n; i++) {       /* 0. eleman zaten aday */
19          if (puanlar[i] > puanlar[enb]) {
20              enb = i;
21          }
22          if (puanlar[i] < puanlar[enk]) {
23              enk = i;
24          }
25      }
```

i	başta	1	2	3	4	5
puanlar[i]	–	91	77	91	58	85
enb	0	1	1	1	1	1
enk	0	0	0	0	4	4

GERÇEK ÇIKTI

```
En büyük: 91 (indis 1, 2. sırada)
En küçük: 58 (indis 4, 5. sırada)
Örnek 4: 6 değeri 12 atışta 5 kez geçti.
```

Örnek 6 ★★: değer var mı, nerede? Doğrusal tarama



Örnek 7 ★★: yerinde ters çevirme — iki uç tekniği

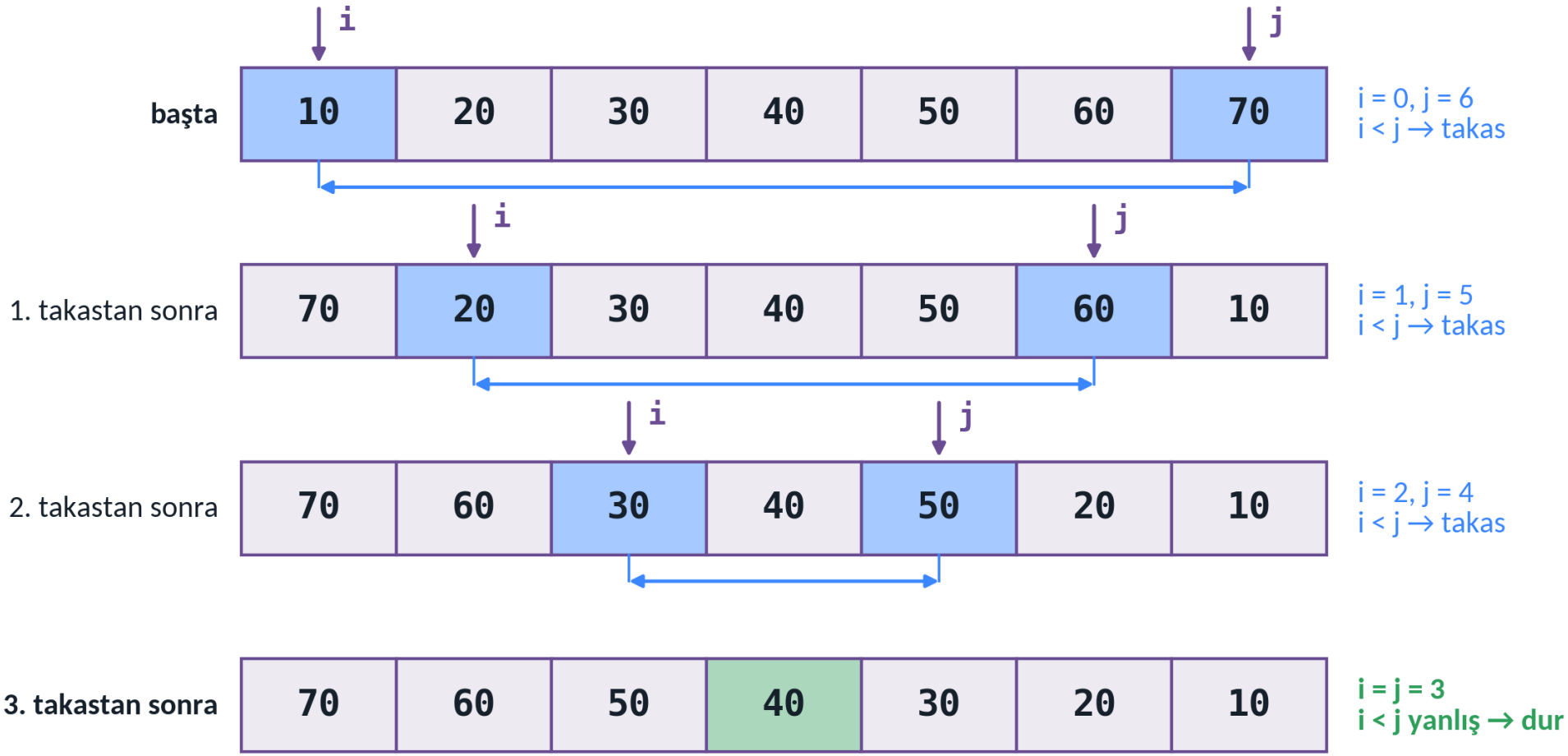
H10_07_ters_cevir.c · satır 23–32

```
23      int i = 0;                          /* sol uç */
24      int j = n - 1;                      /* sağ uç */
25      while (i < j) {                     /* uçlar ortada buluşana kadar */
26          printf("  takas: a[%d]=%d <-> a[%d]=%d\n", i, a[i], j, a[j]);
27          int gecici = a[i];              /* üç atamalı takas */
28          a[i] = a[j];
29          a[j] = gecici;
30          i++;
31          j--;
32      }
```

SINIR TESTLERİ: H10_K13_TERS_CEVİR_TEST.C (GERÇEK ÇIKTI)

```
n = 0: [] -> []  (0 takas)
n = 1: [1] -> [1]  (0 takas)
n = 2: [1 2] -> [2 1]  (1 takas)
n = 5: [1 2 3 4 5] -> [5 4 3 2 1]  (2 takas)
n = 6: [1 2 3 4 5 6] -> [6 5 4 3 2 1]  (3 takas)
```

Örnek 7: i ve j okları, adım adım



TAHMİN ET · TERS ÇEVİRME KOŞULU

Program ne yazdırır?

quiz/H10_Q15_iki_kez_ters.c · satır 7–18

```c
int main(void)
{
    int a[4] = {1, 2, 3, 4};
    for (int i = 0; i < 4; i++) {       /* i < j değil, i < n! */
        int j = 3 - i;
        int g = a[i];
        a[i] = a[j];
        a[j] = g;
    }
    printf("%d %d %d %d\n", a[0], a[1], a[2], a[3]);
    return 0;
}
```

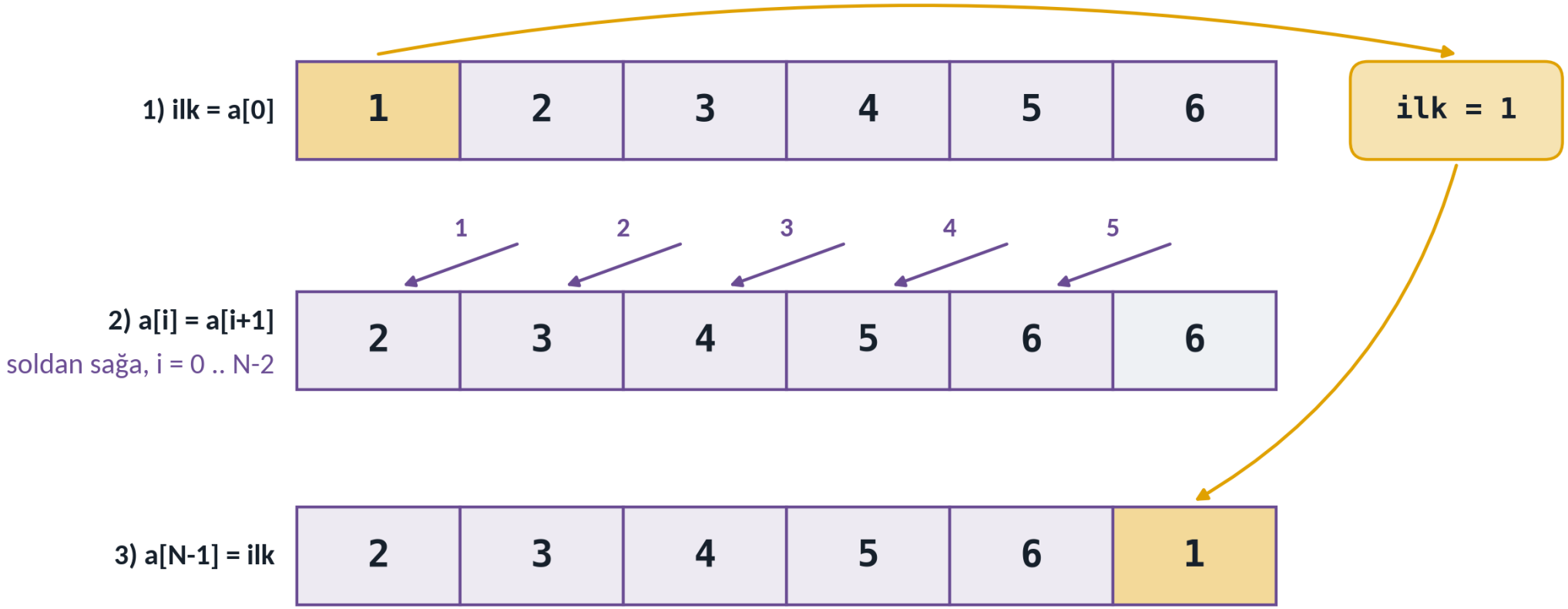
A 4 3 2 1

B 1 2 3 4

C 4 2 3 1

D 1 3 2 4

Örnek 8 ★★: bir konum sola döndürme



Örnek 9 ★★: 6000 zar atışı — frekans dizisi

H10_09_zar_histogram.c · satır 25–30

```c
25      int frekans[7] = {0};              /* [1]..[6] kullanılır, [0] boş */
26
27      for (int atis = 0; atis < ATIS; atis++) {
28          int zar = rand() % 6 + 1;      /* 1..6 */
29          frekans[zar]++;                /* DEĞER = İNDİS */
30      }
```

GERÇEK ÇIKTI (-DSABIT_TOHUM=2026; HER * = 20 ATIŞ)

```
Yüz  Adet  Histogram (* = 20 atış)
  1  1010  **************************************************
  2  1002  **************************************************
  3   965  ************************************************
  4  1057  ****************************************************
  5   988  *************************************************
  6   978  ************************************************
Toplam: 6000 atış (beklenen her yüz için 1000)
```

Frekans dizisinden histogram çubuklarına

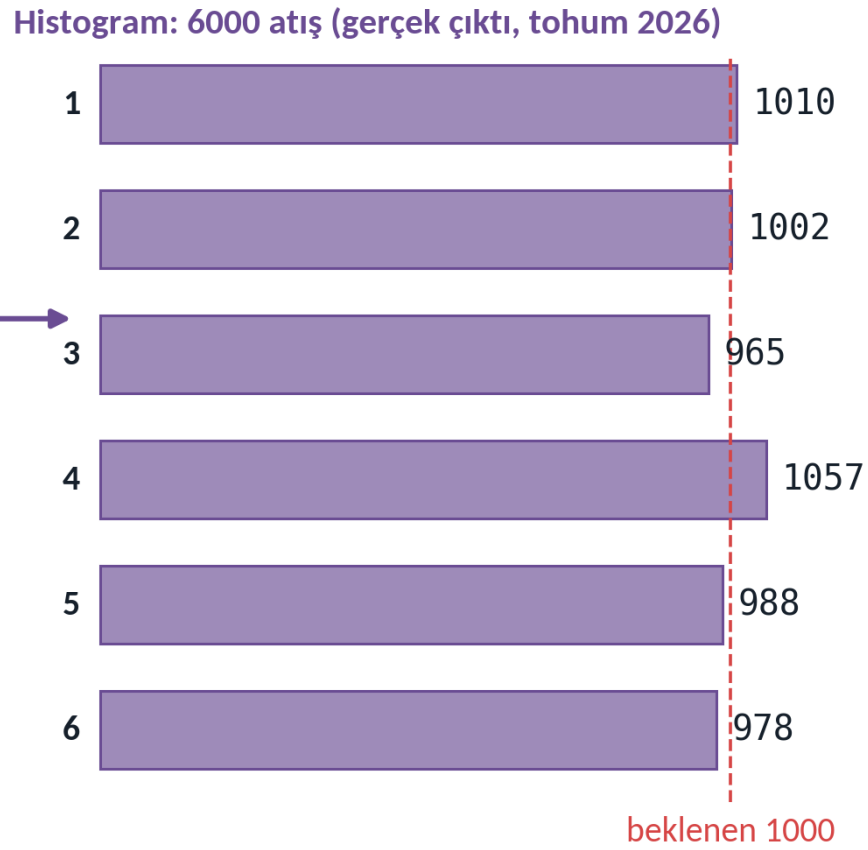
`frekans[zar]++  →  int frekans[7]: değer = indis`

[0]	[1]	[2]	[3]	[4]	[5]	[6]
–	1010	1002	965	1057	988	978

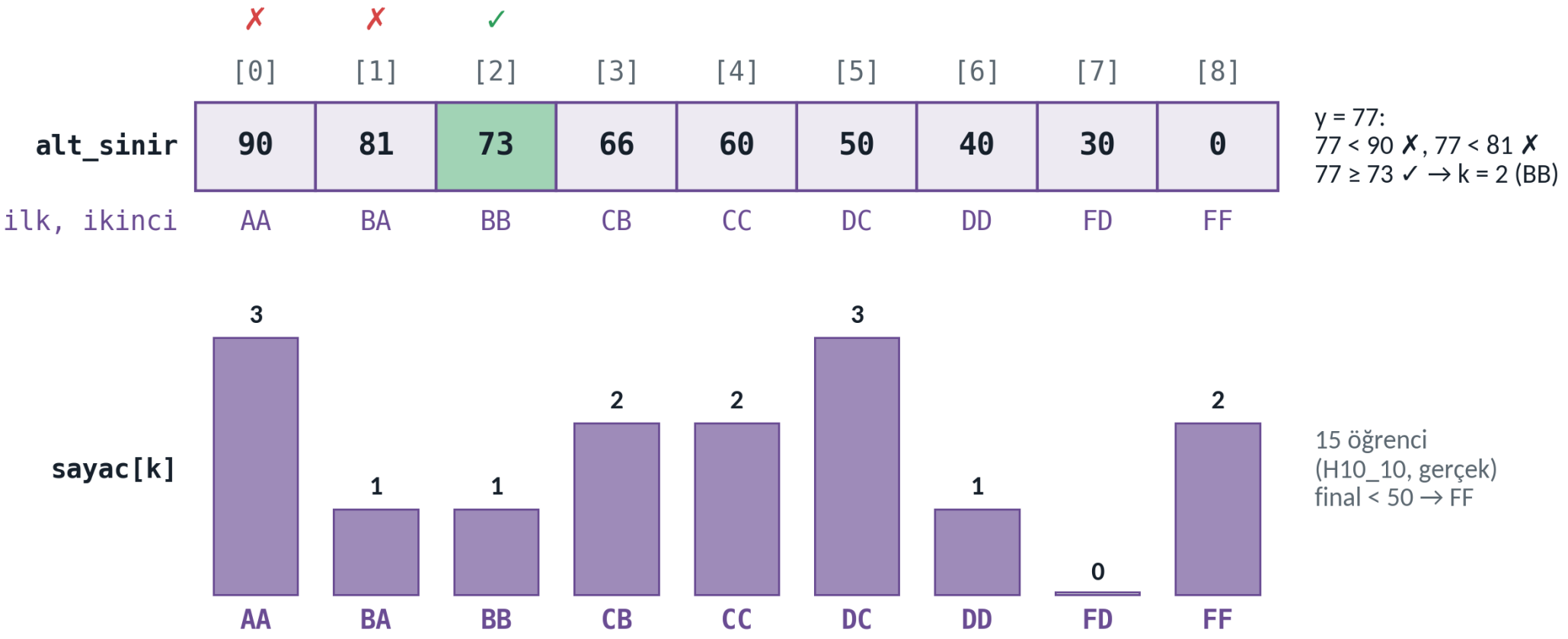
boş

```
int zar = rand() % 6 + 1;
frekans[zar]++;

6 ayrı sayaç + switch (H8) → tek satır
```



Örnek 10 ★★: harf notu dağılımı — frekans + karar



y = 77:
77 < 90 ✗, 77 < 81 ✗
77 ≥ 73 ✓ → k = 2 (BB)

15 öğrenci
(H10_10, gerçek)
final < 50 → FF

Örnek 11 ★★: dizi fonksiyonları ve const

H10_11_dizi_fonksiyonlari.c · satır 15–18

```c
15  bool dizi_oku(int a[], int n);
16  void dizi_yazdir(const int a[], int n);
17  int dizi_toplam(const int a[], int n);
18  int dizi_max(const int a[], int n);
```

H10_11_dizi_fonksiyonlari.c · satır 73–82

```c
73  int dizi_max(const int a[], int n)
74  {
75      int enb = a[0];
76      for (int i = 1; i < n; i++) {
77          if (a[i] > enb) {
78              enb = a[i];
79          }
80      }
81      return enb;
82  }
```

GERÇEK ÇIKTI

```
[12, 7, 25, 3, 25, 9]
Toplam = 81, en büyük = 25
```

CONST İHLALİ (H10_K12)

```
error: assignment of
read-only location ‘*(a + … * 4)’
```

- Dizi + **n** ayrı
- Okuyan: `const`
- `dizi_max`: n ≥ 1, başlangıç `a[0]`

Örnek 12 ★★★: araya ekleme ve silme — kaydırma

H10_12_ekle_sil.c · satır 39–49

```c
int ekle(int a[], int n, int kapasite, int konum, int deger)
{
    if (n >= kapasite || konum < 0 || konum > n) {
        return -1;
    }
    for (int i = n; i > konum; i--) {   /* SONDAN BAŞA: sağa kaydır */
        a[i] = a[i - 1];
    }
    a[konum] = deger;
    return n + 1;
}
```

H10_12_ekle_sil.c · satır 57–59

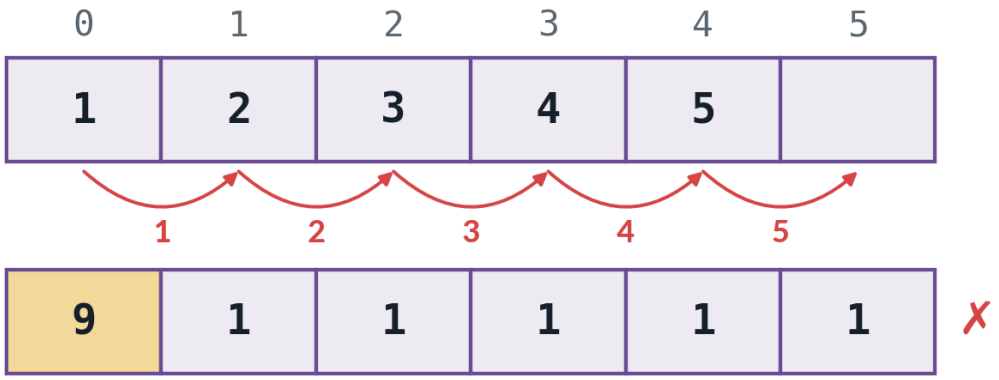
```c
    for (int i = konum; i < n - 1; i++) {   /* BAŞTAN SONA: sola kaydır */
        a[i] = a[i + 1];
    }
```

çağrı	yeni n
ekle(2, 25)	6
sil(0)	5
ekle(n, 60)	6
ekle(0, 5)	-1
sil(7)	-1

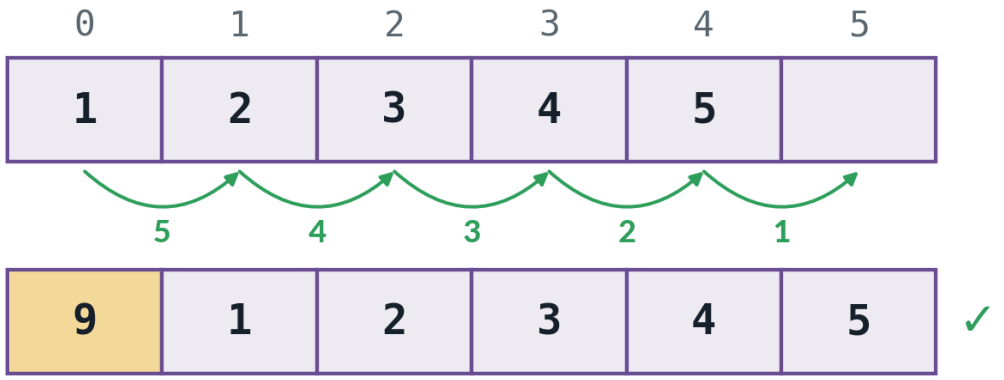
- Önce **denetim**: yer var mı, konum geçerli mi?
- n'yi döndür (değerle aktarım)
- −1: dolu / geçersiz

Kaydırma yönü: neden sondan başa?

EKLEME (başa 9): baştan sona ✗

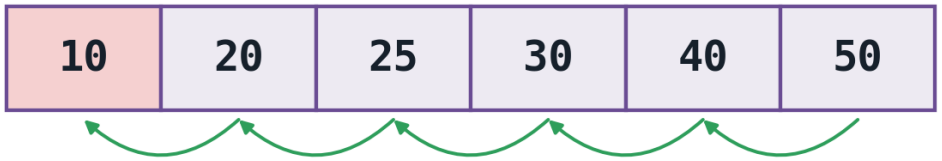


EKLEME (başa 9): sondan başa ✓



Sayılar okların sırası. Baştan sona giderken a[1] = a[0] yazıldığı anda 2 kaybolur: 1 bütün diziye yayılır.

SİLME (konum 0): sola kaydırma baştan sona ✓ (H10_12_ekle_sil.c: sil(0) → [20, 25, 30, 40, 50])



10 silinir; 20, 25, … birer sola kayar; n = 5

Örnek 13 ★★★: sınır dışı yazma deneyi

H10_13_sinir_disi.c · satır 20–22

```
20      int once = 111;                     /* dizinin komşuları */
21      int a[N] = {10, 20, 30, 40};
22      int sonra = 222;
```

H10_13_sinir_disi.c · satır 35

```
35      a[k] = deger;                       /* SINIR DENETİMİ YOK */
```

GERÇEK ÇALIŞTIRMALAR (DEĞER 77; UYARI YOK: -WALL -WEXTRA -PEDANTİC, -O2 BİLE)

```
k =  4: a = {10, 20, 30, 40} once = 111 sonra = 222
k =  5: a = {10, 20, 30, 40} once = 111 sonra = 222
k = -4: a = {10, 20, 30, 40} once = 77 sonra = 222
k = -3: a = {10, 20, 30, 40} once = 111 sonra = 77
k =  6: a = {10, 20, 30, 40} once = 111 sonra = 222  *** stack smashing detected ***: terminated  çıkış durumu: 134
```

Neden? main'in yığın çerçevesi (gcc -S)

yüksek adres ↑

rbp+8	**dönüş adresi**	`a[10], a[11]`	**a[10] = 77 → Segmentation fault (çıkış 139): dönüşte çöktü**
rbp	**saklanan rbp**	`a[8], a[9]`	**a[8] ya da a[9] = 77 → bu çalıştırmada görünür etki yok**
rbp-8	**kanarya (koruyucu)**	`a[6], a[7]`	**a[6] = 77 → *** stack smashing detected *** (çıkış 134)**
rbp-16	**(boş: hizalama)**	`a[4], a[5]`	a[4] ya da a[5] = 77 → görünür etki YOK (sessiz)
rbp-32	**`int a[4]`**	`a[0] .. a[3]`	geçerli indisler: 0 ≤ k < 4
rbp-36	**`deger`**	`a[-1]`	a[-1], a[-2] = 77 → görünür etki yok
rbp-40	**`k`**	`a[-2]`	
rbp-44	**`sonra = 222`**	`a[-3]`	**a[-3] = 77 → sonra = 77 (komşu bozuldu)**
rbp-48	**`once = 111`**	`a[-4]`	**a[-4] = 77 → once = 77 (komşu bozuldu)**

düşük adres ↓

gcc 13.3 -O0 · -fstack-protector-strong (bu GCC'de varsayılan) · konumlar gcc -S çıktısından

Başka derleyici ya da ayarda yerleşim FARKLIDIR: sonuç tanımsız davranıştır.



MÜHENDİS GÖZÜYLE

Yakalamak: AddressSanitizer ilk yazmada durdurur

GERÇEK ASAN RAPORU (KISALTILMIŞ)

```
$ gcc -std=c17 -g -fsanitize=address H10_13_sinir_disi.c -o sinir_disi_asan
$ echo '4 77' | ./sinir_disi_asan
ERROR: AddressSanitizer: stack-buffer-overflow
WRITE of size 4
#0 in main H10_13_sinir_disi.c:35
#0 in main H10_13_sinir_disi.c:19
[96, 112) 'a' (line 21)  ← offset 112: taşma
```

- ASan her yerel dizinin çevresine **yasak bölge** koyar ve her erişimi denetler (Serebryany vd., 2012): `a[4]`'e ilk yazmada satır 35'i gösterir
- Yığın koruyucusu (kanarya), ASLR, yürütülemez yığın: saldırıyı **zorlaştırır**, sınır denetiminin yerini **tutmaz**
- Kural (CERT C ARR30-C): dışarıdan gelen her indisi `0 <= k && k < n` ile denetle — Alıştırma 10.12

Örnek 14 ★★★: Eratosthenes Kalburu (meydan okuma)

H10_14_eratosthenes.c · satır 35–42

```
35      for (int p = 2; p <= ust / p; p++) {        /* p * p <= ust */
36          if (asal[p]) {
37              for (int kat = p * p; kat <= ust; kat += p) {
38                  asal[kat] = false;      /* p'nin katı: asal değil */
39                  isaretleme++;
40              }
41          }
42      }
```

Üst sınır	Asal sayısı	H7 √n testi: bölme	Kalbur: işaretleme
1 000	168	2 351	1 411
10 000	1 229	55 958	16 981
100 000	9 592	1 345 442	193 078

Kalbur 1–100: renk = en küçük asal çarpan

1	**2**	**3**	4	**5**	6	**7**	8	9	10
11	12	**13**	14	15	16	**17**	18	**19**	20
21	22	**23**	24	25	26	27	28	**29**	30
31	32	33	34	35	36	**37**	38	39	40
41	42	**43**	44	45	46	**47**	48	49	50
51	52	**53**	54	55	56	57	58	**59**	60
61	62	63	64	65	66	**67**	68	69	70
71	72	**73**	74	75	76	77	78	**79**	80
81	82	**83**	84	85	86	87	88	**89**	90
91	92	93	94	95	96	**97**	98	99	100

- asal (25 tane, gerçek çıktı)
- 1: tanım gereği asal değil
- 2'nin katları: 4'ten başlayarak
- 3'nin katları: 9'ten başlayarak
- 5'nin katları: 25'ten başlayarak
- 7'nin katları: 49'ten başlayarak

p ≤ 100 / p: yalnızca 2, 3, 5, 7 kalburlanır

104 işaretleme · bölme YOK

bool asal[101]: asal[i] = "i asal mı?"

03

Hata Avcısı, proje ve kapanış

a[n] · ilk değersiz dizi · sizeof tuzağı · = ve == · scanf & · Mini-OBS v0.10

HATA AVCISI

Hata Avcısı 1–2: a[n] ve ilk değeri olmayan dizi

HA1 · `for (int i = 0; i <= N; i++)`

GERÇEK

```
$ ./ha1          # -O0: uyarı yok
Toplam = 33134 (beklenen 369)
$ gcc ... -O2
warning: iteration 5 invokes
  undefined behavior
$ ./ha1_asan
READ of size 4
```

HA2 · `int frekans[7];` (= {0} yok)

GERÇEK (-O0)

```
önceki iş bitti (son değer 1007)
yüz 1: 1002 kez
yüz 2: 1003 kez
yüz 3: 1004 kez
yüz 4: 1004 kez
yüz 5: 1005 kez
yüz 6: 1009 kez
```

- **Çözüm 1:** `i < N` — geçerli indisler 0 … N − 1 · **Çözüm 2:** `int frekans[7] = {0};`
- HA2'de sayaçlar yığında `onceki_is`'in bıraktığı 1000'lerden başladı; `-O2` sürümü "doğru" çıktı verdi: **şansa güvenmeyin**

Hata Avcısı 3–5: gerçek derleyici iletileri

Hata	Gerçek ileti / belirti (GCC 13.3)	Çözüm
`HA3 · fonksiyonda sizeof(dizi)`	warning: ‘sizeof’ on array function parameter ‘dizi’ will return size of ‘int *’ [-Wsizeof-array-argument] · n = 2, ortalama 78.50	n’yi ayrı parametre yap
`HA4a · yedek = notlar;`	error: assignment to expression with array type	eleman eleman döngü
`HA4b · if (a == b)`	warning: comparison between two arrays [-Warray-compare] · “farklı”	eleman eleman karşılaştır
`HA5 · scanf("%d", a[i])`	warning: format ‘%d’ expects argument of type ‘int *’, but argument 2 has type ‘int’ [-Wformat=] · Segmentation fault (139)	scanf("%d", &a[i])

MÜHENDİS GÖZÜYLE

Sınırı programın kendisi bilmeli

- **Her dış indisi denetle:** `0 <= i && i < n` (CERT C ARR30-C); Heartbleed bu kuralın ihlaliydi
- **Kapasite tek yerde, doluluk ayrı:** `#define KAPASITE 100` + `int n;` eklemeden önce `n < KAPASITE`
- **Sınır testleri:** n = 0, 1, kapasite − 1, kapasite; ilk/son konum; bulunamayan değer
- **Araçlar:** `-Wall -Wextra` + `-fsanitize=address,undefined` geliştirirken; uyarı = hata
- **Taşınabilirlik:** `sizeof` kullan, VLA'ya güvenme (MSVC), büyük diziyi yığına koyma

H10_12_ekle_sil.c · satır 41–43

```
41      if (n >= kapasite || konum < 0 || konum > n) {
42          return -1;
43      }
```

GERÇEK: GEÇERSİZ İSTEK, DOKUNULMADI

```
ekle(0, 5) : -1 (dizi dolu)
sil(7) : -1 (geçersiz konum)
```

Heartbleed'in düzeltmesi de tek bir `if` idi: erişimden önce denetim (OpenSSL 1.0.1g).

Mini-OBS v0.10: paralel dizilerde kayıt, istatistik, histogram

main.c — menü döngüsü + VERİ

```
long long no[AZAMI_OGRENCI];
double vize[AZAMI_OGRENCI];
double final[AZAMI_OGRENCI];
double ort[AZAMI_OGRENCI];
int n = 0;
```

1) Not girişi → numara_bul (tekrar yok)

2) İstatistikler

3) Listele

4) Histogram

5) Çıkış

adres + n

obs.c — dizi + n alan fonksiyonlar

```
numara_bul(no, n, x) → indis / -1
dizi_ortalama(ort, n)
standart_sapma(ort, n)  · -lm
en_buyuk_indis / en_kucuk_indis
esik_ustu_say(ort, n, ortalama)
harf_dagilimi(ort, final, n, sayac)
istatistik_yaz · kayitlari_listele
histogram_yaz
```

Mini-OBS v0.10: istatistikler ve kabul ölçütleri

T1: 5 ÖĞRENCİ (GERÇEK ÇIKTI)

```
-------------- SINIF ÖZETİ --------------
Öğrenci sayısı   : 5
Sınıf ortalaması :  68.80
Standart sapma   :  16.08
En yüksek        :  93.00 (2301010047)
En düşük         :  46.80 (2301010046)
Ortalama üstü    : 2 / 5
Başarılı (CC+)   : 3 / 5
Şartlı (DC)      : 1 / 5
-----------------------------------------
```

x_i	$x_i - \mu$	$(x_i - \mu)^2$
79,00	10,20	104,04
46,80	-22,00	484,00
93,00	24,20	585,64
58,00	-10,80	116,64
67,20	-1,60	2,56
μ = 68,80	**Σ**	**1292,88**

$\sigma = \sqrt{(1292{,}88 / 5)} = \sqrt{258{,}576} \approx$ **16,08** ✓

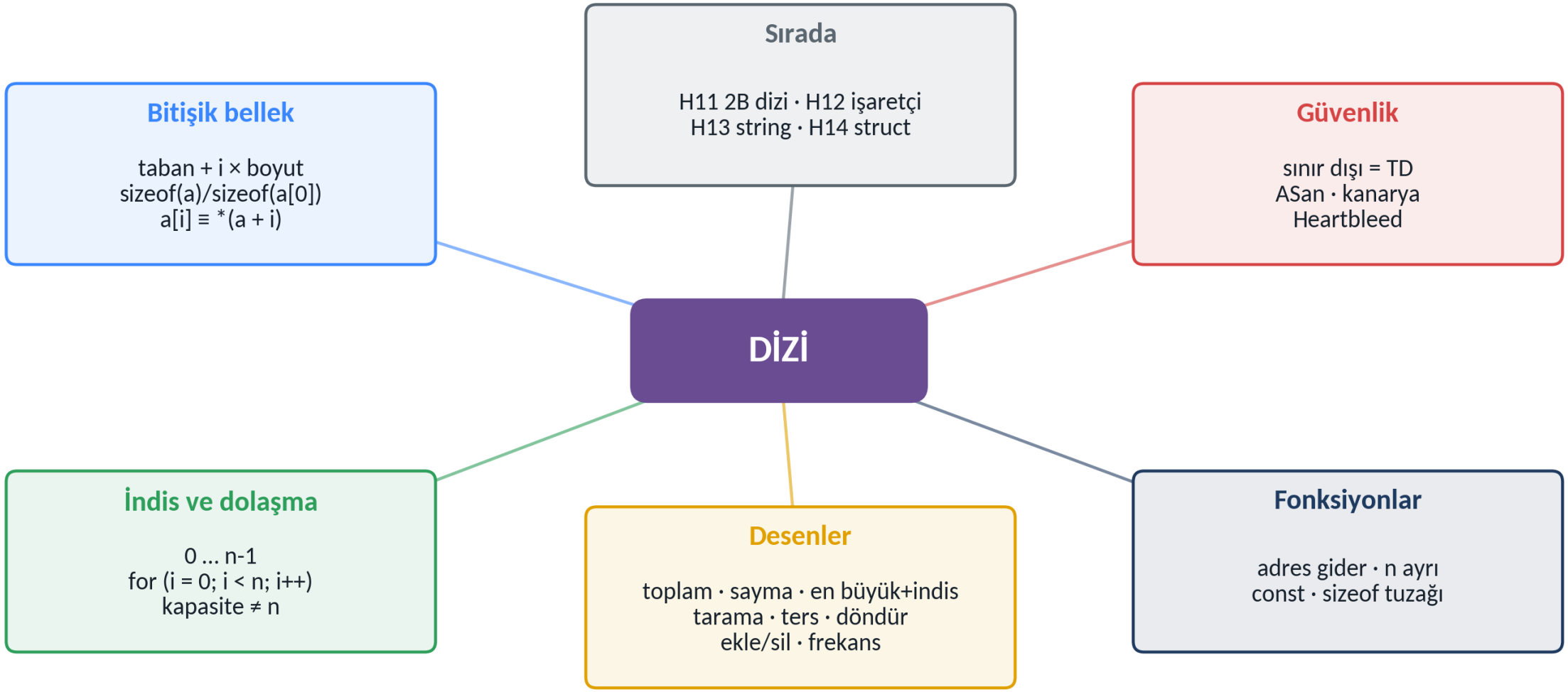
- `make` sıfır uyarı · `make test`: **53 doğrulama** (37 kural + 16 dizi) · bağlarken `-lm` (sqrt)
- 7 senaryo: normal 5 öğrenci, aynı numara, geçersiz girdiler, kayıt yokken 2/3/4, EOF, sınır değerler, kapasite dolu (101. öğrenci)

AP-2 KÖPRÜSÜ

Bu haftanın yapı taşları AP-2’de

- Doğrusal tarama deseni, AP-2’de doğrusal aramanın biçimsel analizine (en iyi, en kötü ve ortalama durum) dönüşür.
- En küçük elemanın indisini bulma + takas, seçme sıralamasının neredeyse tamamıdır.
- İki uç tekniği, AP-2’de iki işaretçi (two pointers) yöntemine; frekans dizisi, sayma sıralamasına (counting sort) temel oluşturur.
- Eratosthenes Kalburunun ve aramaların **biçimsel çözümlemesi** AP-2’de; burada yalnızca işlem saydık

Özet: Hafta 10 kavram haritası



ÇIKIŞ BİLETİ

Dersten çıkmadan önce üç soru

1. `double d[10]` dizisinin taban adresi 0x2000 ise `&d[3]` kaçtır?

2. `int dizi_toplam(const int a[], int n)`: eleman sayısı neden ayrı parametre? `const` neyi garanti eder?

3. Diziye başa eleman eklerken kaydırma döngüsü hangi yönde olmalı? Neden?

Haftaya hazırlık ve kaynaklar

- **H11 — Diziler II: matrisler.** Sınıf listesi, satranç tahtası, fotoğraf = satır × sütun; 2B dizi bellekte yine **tek şerit**
- **Ödev:** Alıştırma 10.7–10.12 (ders notu Bölüm 8); Mini-OBS v0.10 kabul ölçütleri
- **Web:** dizi görselleştirici, histogram oluşturucu, “Sınırı aşarsam ne olur?” ve kalbur simülasyonları
- **Okuma:** King (2008) Bölüm 8; Kernighan & Ritchie (1988) Bölüm 1.6 ve 5.3; Durumeric vd. (2014) Bölüm 2
- **Kodlar:** `kodlar/` (59 .c, çıktılarıyla) · `AP1_H10_kodlar.zip` · ASan ile deneyin